AUTOMATED TRADING PLATFORM PROJECT FINAL

Abdelaziz Hasan

CS 451

GENERAL MOTIVATIONS

In this project, we are aiming to build an automatic trading platform with distributed resources. The final implementation can be expected to include a refined UI for net gains and losses, latency and throughput metrics, both a distributed and a monolithic configuration of the programs, and optionally: subscription to some live feed of data.

CAVEATS

There has generally been no shortage of these. Firstly, this current implementation does not read live data. The heavy requirements of IP management with Fabric's hostile "support" made this difficult. In its place, market data is synthetically generated in the exact form that *websocket*, the module responsible for providing live market data, would send it in. It would be possible to use real market data rather than synthetic data, but from a development perspective, this is very drab. It would be slow to update, and I would be constrained to working in only market hours of the day/week. This is not at all to my benefit

The second setback would be the implementation of a proper strategy. I am not a stockbroker, nor do I have any interest in that matter. The intention behind this project is to apply knowledge of distributed systems to use a collection of resources in a clever way. This does not include learning the nitty gritty on long/short moving averages. As such, the strategy used was grabbed from Stack Overflow as was tweaked very cautiously and faintly for testing purposes only. It is overly simplistic and we will see this as a later issue when we aim to evaluate our monolithic implementation.

The next obstacle is the monitor for the program. After a little thought on what may be a cohesive way to visual the programs out – the very same program that sees no real data and confirms no real trades – would be to simulate net positions and calculate the capital gains and losses for a few different stock options. Again, the data is generated so this information will be completely fictional, but the monitor updates continuously and displays information as it gets it. Thus far, logging efforts have been successful and can be used in the future for a more stylish UI with charts, but that is for another day. Currently, the only results presented are the latest ones. My implementation boasts a distributed configuration as well as a monolithic one, and both have displays in the same UI.

The traditional way to measure latency on a ubuntu machine uses *time.perf_counter()*, a machine specific command that acts as a high precision stopwatch. This fails in a distributed system, each machine has its own counter. Another option had us use *time.time()*, which does not boast the same precision and seems to poorly account for transitions across machines. But, it is *mostly* synchronized which allows us to estimate our latency rather than calculate it. From the ingestion time of our data, carried along through messages, we have the execution and monitor nodes note the time they finish processing the final message in the pipeline. The lack of true synchronization incorporates a negative drift in every latency report; all that is needed is to take the currently observed difference and subtract the minimum raw difference given by the drift. We hold a queue of the hundred most recent fills and monolithic status and average our estimates across all of them to report our relative latency, enqueueing and dequeuing with each new message.

The initial plan to evaluate throughput of the two configurations involved generating a unique token for each portfolio option and managing fifty of them with no delay between posts. In my update, we had only four options and posted two, at random, per second. This proved to be little issue for the monolithic setup, it lacked message passing in between any processes, but the distributed setup had very many issues. Chief among which, the bottleneck introduced by message passing delays any processing and fills message queues with a multitude of outputs. The ingestion node suffered no quarrels as it needed only to start the pipeline, but every node following that introduced a new point of failure. Solving this entailed that I scaled down the problem, include handicaps for ingestion, and enable the different nodes to slow one another down and allow an opportunity to "catch up".

CONFIGURATION

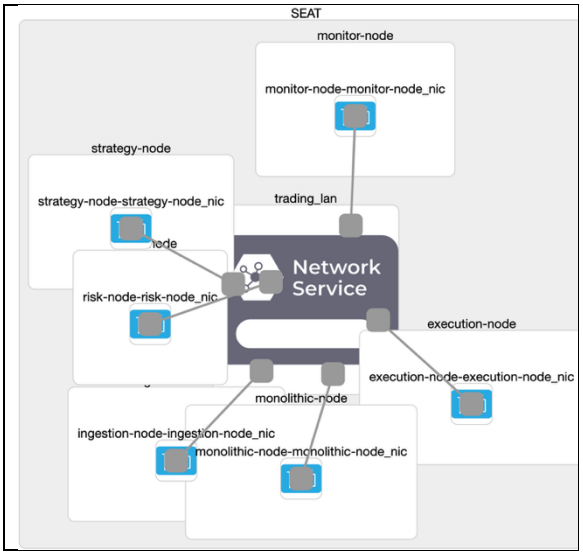

Figure 1: PNG depicting the slice configuration, generated by Fabric.

Models	Usage
class PriceTick(BaseModel): symbol: str price: float timestamp: str ingestion_ts: float = 0.0	Input data – produced in the ingestion node and consumed by the strategy node. Has four channels, one for each mock stock option. Ingestion_ts used for latency.
class Signal(BaseModel): signal_id: str symbol: str action: literal["buy", "sell"] confidence: float price_at_signal: float ingestion_ts: float	After analyzing MAs yielded from PriceTicks, a Signal will be produced if the moving average suggests a 'BUY' or 'SELL' event. Consumed by the risk node. Last two fields are carried throughout.
class Order(BaseModel): order_id: str symbol: str action: literal["buy", "sell"] quantity: int price_at_order: float ingestion_ts: float	Risk node subscribes to Signals, produces to Fills. Confirms a signal based on current position details, e.g. will not short when net position is less than negative five. Last two fields are curried forward.
class Fill(BaseModel): fill_id: str symbol: str action: literal["buy", "sell"] quantity_filled: int fill_price: float ingestion_ts: float	Order confirmation – generally useless in this simulation case but this would be the "receipts" of any confirmed trade by the execution node. Curried price is multiplied by N(1, 0.02) to mimic potential changes between ingestion and execution. Last field will be used for measuring latency.
class MonolithicStatus(BaseModel): portfolio: Dict[str, int] current_prices: Dict[str, float] total_trades: int latest_event: str ingestion_ts: float	Monolithic state used solely for display in the monitor node. Includes all the information relevant to produce an identical output that the distributed dashboard yields, all in a convenient place for analysis.

Figure 2: Chart depicting the model class arrangements and abstract uses.

EXECUTION

Functional nodes include all but 'monitor-node', whose sole purpose is to display data consumed from the Fills and MonolithicStatus bus.

Ingestion: At each market tick, of which there are two per second, a price is generated for all symbols from the distribution U(150, 200) and timestamps are grabbed using python's *time* module. This data is produced to PriceTicks and in the future, we will look into writing PriceTicks with live market data using *websocket*.

Strategy: Subscribed to PriceTicks, every price will be added to a window. With hyperparameters of five and twenty for the short and long window sizes, our node kicks off when twenty ticks are logged. Our small MA looks at the last five ticks in this queue and our long MA looks at the entire twenty. Any more ticks added will dequeue the oldest so that after our length first reaches twenty, we will never have more or less than twenty ticks. If $sMA > lMA * 1.005$, we will issue a "BUY" event to the Signal bus. If $sMA < lMA * 0.995$, we will issue a "SELL" event to the Signal bus. The price and timestamp from the PriceTick message is curried and the symbol is also logged.

Risk: We use simple logic to approve events here. Subscribed to the Signal bus, an Order will be approved only if a "BUY" event occurs when our net position is less than five, and a "SELL" event occurs when our net position is greater than negative five. The idea is that we don't wish to hoard stocks, but this gets complicated when the latency of the program is considered. The distributed dashboard output seems to ignore this logic since messages are stockpiled and concurrently approved. It's a distributed race condition. These Orders are produced and curry forward the ingestion prices and timestamp.

Execution: This node would be responsible for closing trades with some API, but for lack of funds and courage, I left this node a little bleak and have it writing the Fill messages from the subscribed Order messages. All fields are filled accordingly, but the curried price field needs some changes. Ordinarily, this could be grabbed from the trading API as soon as a transaction was closed. We would expect slight differences between the execution price and the ingestion price, due to the latency. To better model this relationship, I multiply the curried ingestion price by N(1, 0.02) to introduce a little market chaos. Now, we can see the changes we expect to observe when trading.

Monolithic: This node is dedicated to hosting the configuration that operates on one machine. All portfolio updates, strategy decisions, risk assessment, and execution is done on this one machine to make any kind of interesting observations we can. Every process is carried out sequentially (from ingestion to execution), and any delays introduced will slow the next portfolio update and action. This node produces to the MonolithicStatus bus, which the monitor node subscribes to for the UI.

DEPLOYMENT

The environment includes python files for five functional nodes, another two for the monitor and models, a *.gitignore* that blacklists both the *project_id* file containing the Fabric *project_id* key and the docx file building this document, and the Jupyter notebook that deploys all the files.

Dependencies & Fabric Configuration: Here we import the modules needed in the notebook, as well as initialize the slice.

Node Configuration: This section allocates some hardware for six different nodes: ingestion, strategy, risk, execution, monolithic, and the monitor node. Early hurdles proved it was beneficial to allocate at least four GB of memory per node. This section also installs the necessary dependencies on each node. The model uses *pydantic* so all nodes need it, as well as *confluent-kafka* and *redis*. My "chaos" with the execution price requires *numpy* on the execution node only. Beyond this, the ingestion node needs to extract files for *kafka* and deploy them before any scripts run, as well as install additional *redis-server* packages. All nodes get their respective python script uploaded here as well.

Deployment: This has been simplified for restarting the program. We open with adjusting the IPs to use internal 10.0.0.* indexed IPs, with the ingestion node at 10.0.0.1, so that all nodes know where to produce and consume events to. Then we deploy a "nuclear clean" procedure, used excessively when experimenting with different strategies in this project. Its role is to shut down/flush out any message bus or server data, as well as removing any scripts or logs from all nodes before reuploading them. We also Then we start *redis* and *kafka* up before running the main script on each node in their backgrounds.

Validation: Here is a little section for peeking at the message bus or the logs on each node, and it even includes the ssh command to log into the monitor. Inside the monitor node, you need to run:

'python3 dashboard.py'

This will start the compact UI that displays the current state. A race condition present in the *kafka* architecture made it unsuccessful to run a background and foreground process of the dashboard UI. Processes under the same group-id will compete for who gets the messages, and this led to an unreliable program. This has since been resolved, and we have shortened our burn-in period from thirty seconds to around three seconds. Running tallies start from zero but log information from messages passed even before the dashboard will boot up.

REMARKS

In the demo, the program was ran with some adjustments to make the two configurations seem a little more evenly matched than they truly are.

1. Distributed ingestion node must limit the number of updates each second.
2. Monolithic node should include a delay for simulated computation.

The intention for both configurations is that updates are posted as fast as the hardware can manage, and they are all processed as fast as they can be. For market updates, unless you are pulling from a ridiculous quantity of portfolio options, you will not run into any of the following issues that will soon be addressed. The ingestion node producing synthetic ticks as fast as the hardware allows falls victim to the inherent latency of passing messages. As fast as the ingestion node works, it fills the PriceTick buffer faster than the strategy node can receive them through the network. The first tweak limits the number of updates per second posted from ingestion to equal the number of portfolio options: twenty. We impose another restriction on the monolithic configuration in the strategy section, though its sequential execution lessens the importance of its placement. Nested in one of the if conditions, we incorporate a delay in the strategy section of our file. This is done with the idea that a monolithic approach will be scaled by the strength of its CPU, whereas a distributed system is scaled by the amount of strategy nodes. The bottleneck in the monolithic system is the CPU itself, and the weakness of the distributed system is the cost of communicating between machines; we need to be able to justify the cost with our payload. Incorporating a delay in the monolithic strategy section simulates a complicated procedure, one that would be justified by distribution. Unfortunately, this delay is only included in the monolithic section because the distributed already struggled to deal with the overflowed buffer of the ingestion node.

With our restrictions in place, our distributed program converges on 450% the throughput and 80% the latency of our monolithic program. TPS is ~280 and ~80 respectively, and latency is 4ms and 5ms respectively. The payload is too simplistic for any adjustments to make a real change in the throughput and latency of the monolithic approach. The limiting factor here is solely the strength of our machine's computing power, whereas our pipeline's sophistication can look beyond any one system's weakness. With the proper distribution of processes to an array of nodes, we can scale this problem indefinitely. The monolithic approach cannot perform as efficiently as the distributed approach, but it is much simpler to interpret and develop. There are zero race conditions with the monolithic approach, unlike those that plague the usefulness of our risk engine procedure. Higher throughput is linked with lower latency, and the distributed setup has the advantage of having an uninterrupted stream of market inputs all the while the strategy, risk, and execution algorithm run. The monolithic approach is disadvantaged with waiting until a market update is processed, assessed, and confirmed before passing another market update. The results explain themselves perfectly.

OUTLOOK

Some things the future may see but won't be included in my final submission:

The strategy performs so poorly, it's almost shameful. In my implementation of a running tally that tracks profit, I was shocked to see a strict negative accumulation of debt. The distributed layout refused to play along with the risk node's ruleset and continuously shorts options while it already has a negative position. Think of it this way, you've ran out of funding and yet you're still putting in orders. It makes no sense and it, unsurprisingly, leads you into further debt. I've already spent too long scratching my head at this, but this could be revisited and the deployment of an entirely new strategy could replace the current one. As I opened with, I am no stockbroker and care only about the distribution of resources to support a program here. Seeing as the monolithic layout performs just as poorly, this is certainly an issue with my choice of strategy and not my implementation. Having any number of strategy nodes would let you plug-and-play whatever procedure you, so long as the *json* messages are received and encoded the same way.

I never got a chance to visit this from the update, but I want to further enhance the UI that's displayed in the monitor node. By reading the last fifty-or-so events in the monitor-node.log, I can make a queue and plot some charts for net gains and losses in each stock option. Paired with the benchmarking, this could present some exciting findings on the martingale nature of the stock market. The complications that arose here include the following: random generation of data does not follow any stochastic process. Resetting and deploying the project files has gotten to be so tedious that I could not find the time to include this in the proper environment. The profit we observe has such a sharp decline that it would be inconceivable to suggest this program to model martingale properties. We need to fix both the strategy, and the data generation process to make any meaningful observations.